

DataLeakDetector h 分支技术工作总结

生成日期: 2026-06-30
分支: h

当前 HEAD: b3fb78d Improve benchmark table category inference
关联报告: reports/introduce.typ、docs/record.md

1.1 摘要

h 分支的工作目标是把 DataLeakDetector 从 “主要依赖录屏/VLM 判断” 的原型链路，推进为 “日志优先、视觉兜底、事件关联、符号推理” 的可评估检测系统。分支同时处理三类问题：第一，降低 VLM 图片调用成本；第二，提升真实办公外发场景的覆盖能力；第三，使项目结构、评估口径和报告输出能够支撑后续数据集实验。

从实现上看，分支形成了三条主线：

- 检测链路增强：新增 log-first 确定性检测、VLM fallback gate、Qwen/VLM 输出鲁棒后处理、Datalog fact 注入。
- 架构层补齐：新增 2-EventCorrelator，承接 docs/introduce.md 中日志挖掘、FrontendApp、敏感窗口、多跳 lineage 和 correlation bundle 职责。
- 评估与报告体系：新增离线 benchmark、NAS benchmark live VLM 验证、论文风格表格生成器，以及本报告和 reports/introduce.typ。

NAS 全量样例 211 全量 VLM benchmark 已完成	Live VLM 复核 146 success=145 / skipped=1	Final F1 63.6% precision=93.3%, recall=48.3%
---	---	--

总体结论

分支已经建立可复用的检测和评估骨架。当前 triage 阶段 recall 为 100%，说明 “日志确定 + 是否送 VLM” 这一层没有漏掉正例；final 阶段 recall 为 48.3%，说明主要瓶颈转移到 VLM 完成态判断和后处理策略。后续工作应从 “是否调用 VLM” 转向 “如何让 VLM 更稳定地确认发送、上传、屏幕共享和内容暴露的完成态”。

1.2 分支演进与提交追踪

本分支的提交可以分成四个阶段：VLM 后处理鲁棒化、log-first 检测与 fallback gate、NAS benchmark 与视觉 anchor、EventCorrelator 架构层与报告输出。

提交	阶段	主题	技术内容
e2dbafe	Log-first	perf: reduce API usage with log-first detection	建立日志优先检测链路；日志证据足够时直接输出上传事件，减少不必要的 VLM 图片调用。
5b5d770	Log-first	fix: handle upload-only and renamed sensitive files	修复只出现上传日志、重命名后上传、派生文件上传时原始敏感文件映射缺失的问题。
dee2003	Fallback gate	Add token-aware VLM fallback scenarios	增加 AI 粘贴、截图、录屏、剪贴板复制等需要视觉兜底的场景，并加入帧数和 token 压力估算。
41d781b	VLM 鲁棒化	Harden VLM event postprocessing for Qwen	增强 Qwen/VLM JSON 解析、字段标准化、重复事件合并和正常阅读噪声过滤。
eef0151	VLM 鲁棒化	Add harder realistic accuracy fixtures	增加更接近真实办公的 fixture，覆盖前置说明 JSON、长文本粘贴、截图外发和正常阅读负例。

c9cea94	缺口覆盖	Add missed violation fixture coverage	补充原链路漏检样例，包括字段别名、语义改写、屏幕共享、二维码、云同步、U盘和 HTTP POST。
a21efe1	缺口覆盖	Improve robust violation detection	将修复抽象为敏感概念组、风险动作词、字段别名、外部通道识别和延迟导出追踪。
87f8aa7	离线评估	Add offline detection benchmark	新增 tools/benchmark_detection.py，输出 precision、recall、F1、失败样例和估算 VLM 调用压力。
6110e6a	NAS 工具	Improve log-first triage and NAS tooling	增强 NAS 样本加载、日志融合、敏感文件发现和 log-first triage 统计。
f1d7bb1	NAS 工具	Tighten log-first triage noise filtering	过滤系统路径、浏览器缓存、监控自身输出等噪声，减少无意义 VLM review reason。
724455e	视觉 anchor	Add visual anchors for VLM triage	为 fallback 构造更明确的视觉复核窗口，保留候选日志上下文和关键帧锚点。
1fa8f84	全量 VLM	Add live VLM verification to NAS benchmark	为 NAS benchmark 增加 --use-vlm、帧抽取、VLM verdict、correlation bundle 和 case 过滤。
2a31428	架构层	Add event correlator architecture layer	新增 2-EventCorrelator，实现 FrontendApp、敏感窗口、多跳 lineage、correlator service 和 demo。
90e338c	报告工具	Add benchmark table reporter	新增 tools/report_benchmark_table.py，把 benchmark JSON 转成论文表格格式。
b3fb78d	报告工具	Improve benchmark table category inference	修正 stage4/e2e、stage5/U* 和 嵌套 session 的类别与应用推断。

1.3 文件级变更追踪

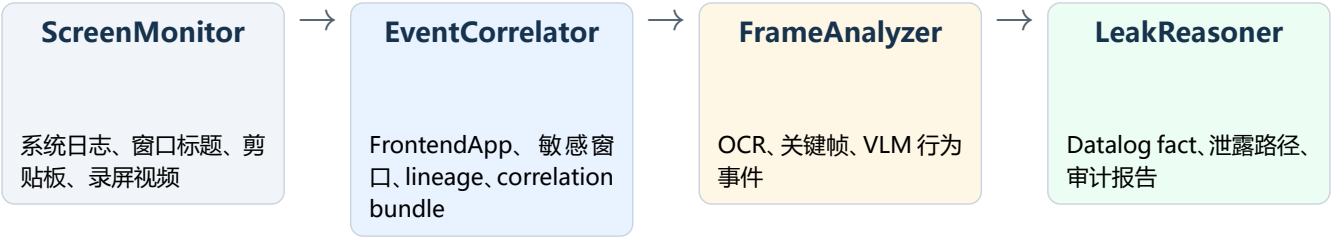
本分支不是单一模块修改，而是围绕“日志优先、视觉兜底、事件关联、可评估输出”形成一组互相支撑的文件级改动。下表按职责列出主要文件、引入原因和当前状态。

文件	职责	实现说明
3-RiskHunter/log_first_detector.py	确定性日志检测	扫描日志中的敏感源、派生文件和外发通道。输出 UploadEvent、operation_records、file_mappings 和统计信息。
3-RiskHunter/suspicious_window_vlm_trigger.py	VLM 触发窗口	根据敏感 anchor、AI/外发/截图/录屏/会议等信号生成 analysis_windows，并过滤白名单和系统噪声。
3-RiskHunter/frontend_app_classifier.py	视觉风险分类	对窗口标题和应用上下文进行类别判断，识别 AI、会议、屏幕共享、远程桌面、外部应用等。
run_e2e.py	端到端编排	在模块 3 前先执行 log-first；若已有确定上传则跳过 VLM，否则调用 fallback gate，再进入原 LangGraph/VLM 链路。
1-FrameAnalyzer/agent.py	VLM 输出解析与过滤	处理 Qwen 返回格式差异、字段别名、敏感概念匹配、风险动作匹配和低价值正常事件过滤。

2-EventCorrelator/ event_correlator/ frontend.py	FrontendApp 解析	将浏览器窗口进一步解析为 email、ai_service、cloud_storage、code_repo、meeting、workplace 等类别。
2-EventCorrelator/ event_correlator/ windows.py	敏感窗口构建	按敏感 anchor、非白名单共现应用和后续外部 FrontendApp 生成可供 FrameAnalyzer 消费的分析窗口。
2-EventCorrelator/ event_correlator/ lineage.py	多跳派生追踪	从敏感 root 扩展到 known artifacts, 导出 artifact_instances, 为同名同路径不同派生实体提供证据 ID。
2-EventCorrelator/ event_correlator/ correlator.py	事件关联	将日志事件、VLM frame segments 和 lineage 合并为 CorrelationBundle, 输出 correlated_events 和 upload_candidates。
tools/ benchmark_nas_samples.py	NAS 全量评估	支持 --use-vlm、--case、VLM verdict、frame segment 包装、EventCorrelator bundle 和 JSON 报告输出。
tools/ report_benchmark_table.py	论文表格生成	将 benchmark JSON 聚合为 Category/#Case/#Apps/#Held/Prec/Recall/F1 格式。
reports/introduce.typ	项目介绍报告	说明项目定位、架构映射、数据流、VLM 触发逻辑、全量评估结果和迁移计划。

1.4 架构对齐

docs/introduce.md 中描述的目标架构不是旧目录编号意义上的“模块 1/2/3”，而是按职责划分的感知与推理链路。当前仓库仍保留历史目录，以避免破坏已有脚本；分支通过新增模块和报告说明完成过渡。



当前目录	目标职责	说明
ScreenMonitor/	证据采集	负责采集文件事件、窗口切换、剪贴板、截图/录屏等原始信号。
2-EventCorrelator/	日志挖掘与事件关联	新增架构层，负责前台网页分类、敏感窗口、多跳派生链、日志和 VLM segment 的关联。
1-FrameAnalyzer/	视频关键帧与 VLM	历史目录名保留。目标职责是窗口内抽帧、OCR、关键帧选择和 VLM 行为事件输出。
2-FileTracker/	旧派生追踪原型	保留旧行为图和 worklist。新 lineage 能力逐步迁移到 EventCorrelator。
3-RiskHunter/	风险编排和 log-first	当前主入口之一，包含 LogFirstDetector、VLM fallback gate、suspicious windows。
4-ThreatDetector/	符号推理	将上游事件转为 Datalog fact，并输出可解释泄露路径。
tools/	评估与数据工具	NAS benchmark、离线 benchmark、表格生成、数据下载。
reports/	报告和实验产物	Typst 报告、PDF、NAS 中间结果和可审计实验记录。

1.5 文件架构处理

本轮没有强行移动顶层目录。该决策基于两点：第一，run_e2e.py、NAS benchmark 和多个模块存在硬编码 sys.path；第二，前一轮全量 VLM 评估已经形成可比较基线，目录大迁移会引入不必要的确定性。

实际完成的文件架构处理包括：

- 将新增的事件关联层统一命名为 2-EventCorrelator，避免继续使用 02-EventCorrelator 的 leading-zero 风格。
- 清理新增模块和全仓验证过程中产生的 __pycache__。
- 在 2-EventCorrelator/README.md 中说明其职责来自 docs/introduce.md 的 EventCorrelator，而不是旧 FileTracker 编号。
- 新增 reports/introduce.typ，系统介绍项目架构、数据流、VLM 触发、全量评估和迁移计划。
- 本报告重写为当前分支级工作总结，和 introduce.typ 在术语、指标、目录映射和下一步计划上保持一致。

后续目录迁移建议

若需要彻底统一目录结构，应先增加稳定包入口，例如 modules/event_correlator、modules/frame_analyzer、modules/leak_reasoner，再逐步替换脚本中的 sys.path.insert。在迁移完成前，保留历史目录更有利于维持 benchmark 可运行和指标可对比。

1.6 实现细节一：Log-first 确定性检测

LogFirstDetector 的目标是在日志证据足够时直接给出外发结论，避免把确定性问题交给 VLM。实现位置为 3-RiskHunter/log_first_detector.py。

核心状态结构：

- source_by_key：记录已确认属于敏感链路的文件路径，以及其原始敏感源。
- mappings：记录派生文件到原始敏感文件的映射。
- operation_records：记录打开、转换、上传等可审计操作。
- upload_events：最终输出的统一外发事件。

主循环逻辑：

```
for log in logs_by_time:
    path = normalize_path(log.get("file_path", ""))
    detected_original = self._upload_detection_original_file(log)

    if self._is_sensitive_path(path, log):
        source_by_key[file_key(path)] = {...}

    parent = self._find_parent_for_log(log, source_by_key, mappings)
    if parent:
        mappings[file_key(path)] = parent["original_file"]

    if is_upload_log(log):
        event = self._build_upload_event(log, source_by_key, mappings)
```

这段逻辑覆盖以下场景：

场景	判断依据	输出
浏览器临时上传	upload_detection.original_file 指向真实敏感源，当前 file_path 可能只是 .tmp。	将临时文件映射回真实敏感文件。
重命名/压缩/导出	文件名相似、同进程、时间邻近、导出上下文、敏感词相似。	建立派生文件映射链。
云同步	目标路径或窗口文本包含 Dropbox、OneDrive、Google Drive、云盘、同步等 marker。	operation_type=cloud_sync。

U 盘/移动介质	写入 E:/ 等可移动盘路径，或文本包含 removable/usb/u 盘。	operation_type=removable_media。
HTTP POST/API 上传	http_post、http_put、api_upload 或 URL/ POST marker。	operation_type=network_upload。

1.7 实现细节二：VLM fallback gate

VLM 触发逻辑由 run_e2e._should_use_vlm_fallback 和 3-RiskHunter/suspicious_window_builder.py 共同承担。该层不判断最终泄露，只决定是否值得花 VLM 成本。

判断公式可以概括为：

```
should_run_vlm =
    no_deterministic_upload
    and sensitive_context_exists
    and suspicious_visual_or_external_signal_nearby
    and not whitelist_or_system_noise
```

典型触发信号：

- AI 与网页聊天：ChatGPT、Claude、Gemini、DeepSeek、Kimi、Poe、豆包、通义、Copilot。
- 外发动作：clipboard、paste、copy、send、upload、attach、share、mail、drive、http、usb。
- 视觉风险：screenshot、screen recording、screen share、meeting、virtual machine、remote desktop。
- 派生风险：created、modified、renamed、copied、compressed、converted、clipboard_text、screenshot_capture。

当前门控结果：

- triage precision=88.78%
- triage recall=100.00%
- triage f1=94.05%

该结果说明门控倾向于高召回。系统先把可疑样例送入视觉复核，再由 VLM 和后处理提高精度。

1.8 实现细节三：EventCorrelator 架构层

新增 2-EventCorrelator 后，日志挖掘层不再只是 benchmark 内部函数，而是形成可复用模块。

1.8.1 FrontendApp 解析

event_correlator/frontend.py 从窗口标题、URL 和进程名提取前台应用类别。浏览器窗口不再只保留 msedge.exe 或 chrome.exe，而是进一步归类：

```
{
    "window_title": "mail.163.com 和另外 1 个页面 - Microsoft Edge"
}
```

解析结果：

```
{
    "category": "email",
    "display_name": "email:mail.163.com 和另外 1 个页面",
    "is_external": true
}
```

当前规则覆盖 email、ai_service、cloud_storage、code_repo、messaging、meeting、workplace 等类别。该能力直接服务于 analysis_windows 构建和后续图写回。

1.8.2 敏感窗口构建

event_correlator/windows.py 根据敏感文件 anchor 构建窗口。match type 包括：

```
exact_path
filename
window_title
derived_under_sensitive_stem_dir
keyword
```

窗口包含：

- window_id
- sensitive_file
- start
- end
- match_types
- cooccur_apps
- frontend_categories
- candidate_events
- post_buffer_seconds

这与 docs/introduce.md 中 “敏感窗口构建” 的要求保持一致。

1.8.3 多跳 lineage

event_correlator/lineage.py 从一轮派生扩展为沿已知 artifact 集合继续推断。新增 artifact_instances, 用于区分同名同路径但不同时间产生的文件。

smoke 输入：

customer.xlsx -> converted customer.pdf -> compressed customer.zip -> selected in mail

输出：

C:/work/customer.pdf -> C:/work/customer.xlsx
C:/work/customer.zip -> C:/work/customer.pdf
upload_candidates=1

该能力解决 docs/introduce.md 中 “派生文件迭代追踪只做一轮” 的问题雏形。

1.9 实现细节四：NAS live VLM benchmark

tools/benchmark_nas_samples.py 增加 live VLM 流程后，benchmark 具备三层结果：

- deterministic：只统计 log-first 确定性检测。
- triage：统计确定性检测 + 需要 VLM 的候选。
- final：统计 VLM verdict 和 EventCorrelator correlation bundle 后的最终结果。

关键实现：

函数/参数	职责	说明
--use-vlm	启用线上 VLM	使用 .env 中配置的 API key、base URL 和模型。
--case	单 case 过滤	支持调试 stage1/2-ai-poe-1 等单样例。
_review_log_context	补充日志上下文	把窗口内 clipboard、paste、upload、send、mail、AI 等关键日志摘要提供给 VLM。
_live_vlm_review_case	抽帧并调用 VLM	按 fallback windows 抽取关键帧，构造图文 prompt，解析 JSON verdict。
_frame_segments_from_verdict	verdict 转 segment	将 VLM 判断包装成 EventCorrelator 可消费的 frame segment。
_run_event_correlator	最终关联	通过 2-EventCorrelator 输出 upload_candidates，作为 final positive 判断依据。

1.10 实现细节五：表格生成器

tools/report_benchmark_table.py 将 benchmark JSON 转换为论文风格表格。输入为 output/nas_full_vlm_report.json，输出为：

- output/nas_full_vlm_table.md

- output/nas_full_vlm_table.json
- 标准输出 markdown table

类别推断考虑了本地数据集的不规则命名：

- 0-normal-email-* 作为 held-out/正常应用覆盖。
- stage4/e2e-* 和数字样本归为 E2E。
- stage5/U1-U5 映射为 Steganography、Annotation、Virtual Machine、Bluetooth、Cloud Drive。
- 嵌套 session_* 目录不会被误识别为类别。

该工具用于复现 docs/image.png 所示的论文表格格式。

1.11 实现细节六：VLM 后处理鲁棒化

VLM 后处理的目标不是“相信模型每一句话”，而是把模型输出转换为稳定、可审计、可回归测试的结构化事件。该部分主要在 1-FrameAnalyzer/agent.py 中实现，覆盖解析、归一化、过滤、去重和关键词匹配。

1.11.1 返回格式解析

Qwen/VLM 常见返回格式包括：

- 直接返回 JSON 数组。
- 使用 JSON fence 包裹。
- 在 JSON 前后添加解释性文本。
- 返回单个对象而非数组。
- 字段缺失或使用近义字段名。

后处理链路先从响应中提取 JSON 片段，再转换为事件列表。若模型返回单个对象，则包装为单元素数组；若返回空或不可解析内容，则记录失败原因，而不是让主流程崩溃。

典型输入：

```
根据图片判断，存在如下风险事件：
JSON 片段：
[
  {"operation": "添加邮件附件", "file_name": "客户名单.xlsx"}
]
```

标准化后：

```
{
  "operation_type": "添加邮件附件",
  "original_filename": "客户名单.xlsx"
}
```

1.11.2 字段别名归一化

VLM 并不稳定遵循固定 schema，因此后处理会把多个别名收敛到统一字段：

标准字段	常见别名	用途
operation_type	operation, action, 行为, 操作	判断是否为上传、发送、截图、复制、屏幕共享等风险动作。
original_filename	file_name, filename, source_file, sensitive_file	与敏感文件、OCR 文本和日志路径进行匹配。
target_file	derived_file, output_file, target_path	用于识别派生文件和构建 lineage。
app_name	active_app, application, window_app	用于判断外部 sink 或白名单应用。
time_range	timestamp, time, frame_time	用于证据排序和 Datalog fact 时间戳。

1.11.3 敏感概念组

直接字符串匹配不足以处理 VLM 的语义改写。例如敏感文件名是“薪资表”，模型可能描述为“薪酬明细”“工资数据”“员工收入”。因此分支引入概念组匹配：同一概念组内的词可以互相支持。

示例概念组：

薪资：薪资，薪酬，工资，salary，payroll
客户：客户，client，customer，名单，联系方式
预算：预算，budget，财务，成本，董事会
合同：合同，协议，contract，agreement
账号：账号，密码，credential，token，key

匹配策略是分层的：

1. 精确文件名或路径命中。
2. 文件名去符号后的模糊匹配。
3. 敏感概念组共享。
4. 风险动作词补强。

只有“敏感概念 + 风险动作”同时成立时，才会保留缺少精确文件名的 VLM 事件，从而降低正常阅读误报。

1.11.4 低价值正常事件过滤

模型经常把普通查看、阅读、滚动也输出成事件。这类事件不能直接作为外发证据。后处理会过滤以下低价值事件：

- 打开文件但没有复制、截图、上传、发送等动作。
- 阅读文档、滚动页面、切换窗口。
- 普通编辑或本地保存。
- 白名单应用内的本地操作。

保留条件通常需要满足至少一个风险动作，例如：

paste / copy / send / upload / attach / screenshot / recording
screen share / qr / encode / export / cloud sync / removable media

1.11.5 去重与合并

同一段画面中，模型可能对连续帧输出重复事件。后处理按文件名、应用、操作、时间桶进行去重，并合并 evidence refs。这样既保留证据来源，又避免一段上传被重复计数。

1.12 实现细节七：派生映射评分

派生文件追踪是系统准确率的关键。真实场景中敏感文件不会总是以原名上传，常见路径包括：

薪资.xlsx -> 薪资.zip -> 邮件附件
客户名单.docx -> 客户名单.pdf -> 网盘上传
预算.xlsx -> notes.pdf -> ChatGPT/邮箱
customer.txt -> customer.zip -> customer.dll -> customer.zip -> U盘

LogFirstDetector._find_parent_for_log 和 EventCorrelator.LineageBuilder 使用相似但互补的策略：

信号	含义	作用
stem 相似	当前文件名和父文件名存在前缀、包含或去扩展名相似。	处理重命名、压缩、另存。
同进程短窗口	同一进程在短时间内打开源文件并创建目标文件。	处理 Word/Excel 导出、压缩软件生成文件。
导出上下文	窗口或事件中出现 export/save as/convert/pdf/另存/导出。	处理文件名变得不相似的延迟导出。
显式父路径	日志中存在 source_path、original_file、old_path、related_paths。	优先使用强证据，避免启发式误连。
known artifacts	已经确认的派生文件也可作为下一轮父节点。	解决只追一轮的问题。

文件级 lineage 输出包括：

- direct_file_mappings: child -> parent。
- full_file_mapping_chains: root -> ... -> child。
- artifact_instances: path + nearest evidence time。

artifact_instances 的意义在于处理同路径同名但不是同一个文件的情况。若两个 customer.zip 在不同时间由不同父文件产生，路径相同并不代表它们是同一证据实体。

1.13 实现细节八：Datalog 注入与证据链连通

ThreatDetector 不应只接收“某个文件上传了”的孤立结论，而应接收能推理的 fact。run_e2e.py 中的补充 fact 注入会从模块 3 输出中构建以下关系：

```
OpenFile(proc, original_file, ts)
TransferFile(proc, original_file, derived_file, ts)
CrossProcessTransfer(from_proc, to_proc, shared_data, ts)
LeakFile(proc, leaked_file, channel, ts)
```

示例：

```
OpenFile(wps.exe, 员工薪资明细表Q4.xlsx)
TransferFile(wps.exe, 员工薪资明细表Q4.xlsx -> 员工薪资明细表Q4_part1.xlsx)
CrossProcessTransfer(wps.exe -> msedge.exe, 员工薪资明细表Q4_part1.xlsx)
LeakFile(msedge.exe, 员工薪资明细表Q4_part1.xlsx, network)
```

该链路能解释：

- 谁打开了敏感源文件。
- 文件是否经过压缩、拆分、导出或重命名。
- 外发进程和源进程是否不同。
- 最终泄露渠道是网络、邮件、云同步、可移动介质还是屏幕共享。

当前 Datalog 注入仍是轻量实现，主要服务 E2E 报告和 deterministic leak path。后续 Neo4j writer 完成后，应把这些 fact 与图数据库边保持一致。

1.14 典型样例前后对比

以下样例用于说明本分支为何不是按 case id 写死，而是沉淀为可复用规则。

样例	修复前	修复后	关键能力
字段别名	VLM 返回 operation 或 file_name，旧逻辑只认 operation_type 和 original_filename，事件被丢弃。	字段别名归一化后保留事件。	VLM schema normalization
薪资/薪酬改写	关键词是“薪资表”，VLM 写“薪酬明细”或“工资数据”，短字符串匹配失败。	概念组匹配识别为同类敏感对象。	sensitive concept groups
屏幕共享预算	画面没有精确文件名，只出现“董事会预算数字和成本明细”。	预算/财务概念 + 屏幕共享动作共同命中。	risk action + concept
二维码外发	不属于传统上传/发送动作，旧风险词不覆盖。	qr、二维码、编码、生成 纳入风险动作。	risk token expansion
云同步目录	复制到 Dropbox/OneDrive 被当成普通文件创建。	外部目的地识别为 cloud_sync。	external destination
U 盘拷贝	写入 E:/... 不属于 file_upload，被跳过。	可移动介质写入识别为 removable_media。	removable media detection
HTTP POST	http_post 不在上传事件类型中。	NETWORK_UPLOAD_TYPES 覆盖 POST/PUT/API upload。	network upload detection
延迟导出	打开预算表 8 分钟后导出 notes.pdf，文件名不相似且超过短时间窗口。	export/save-as 上下文将其连回敏感源文件。	export context scoring

1.15 抽象能力沉淀

本分支的修复重点不是 “让某个样例过”，而是形成可迁移的检测能力。

能力	抽象规则	覆盖范围
字段标准化	将模型输出、日志字段、upload metadata 映射为统一 schema。	不同模型和不同日志格式
敏感概念组	用概念组补充精确文件名匹配。	语义改写和 OCR 缺字
风险动作词	上传、发送、粘贴、截图、录屏、屏幕共享、二维码、导出、云同步、U 盘。	非传统上传渠道
时间窗口	敏感 anchor 附近的外部前台应用和可疑动作触发 VLM。	控制误报和 token
派生链	强父路径优先，启发式评分兜底，known artifacts 多跳追踪。	重命名、压缩、另存、转换
评估表格	从 benchmark JSON 自动聚合类别级指标。	论文结果和实验复现

1.16 Token 消耗分析

本分支将 VLM 从入口模型调整为兜底模型。成本路径可以分为三类：

路径	触发条件	成本特征
确定性日志路径	日志中已有敏感源、派生链和上传/同步/可移动介质/HTTP POST。	不调用 VLM，成本最低。
VLM fallback 路径	日志不能确认内容，但有敏感上下文和可疑视觉动作。	抽取少量关键帧，调用 VLM 判断内容暴露和完成态。
跳过路径	无敏感上下文、系统噪声、白名单应用、远距离无关窗口。	不调用 VLM，避免负例成本。

全量 NAS 中：

- vlm_reviews=196：trriage 阶段认为需要 VLM 或视觉复核的样例数。
- live_vlm_reviews=146：实际发起 live VLM 的样例数。
- success=145：成功获得 VLM JSON verdict。
- skipped=1：缺少视频、录屏起点或帧抽取失败导致跳过。

这个结果说明 VLM 成本主要集中在日志无法直接确认的模糊外发场景。后续若实现并发 VLM 和更精细的帧选择，可降低运行时间，但不会改变当前成本分层原则。

1.17 评估结果

全量 NAS + live VLM 运行命令：

```
python tools/benchmark_nas_samples.py --use-vm --json-output output/nas_full_vlm_report.json
python tools/report_benchmark_table.py output/nas_full_vlm_report.json --markdown-output output/nas_full_vlm_table.md --json-output output/nas_full_vlm_table.json
```

总体指标：

阶段	TP	FP	TN	FN	Precision	Recall	F1
Triage	174	22	15	0	88.78%	100.00%	94.05%
Deterministic	50	0	37	124	100.00%	28.74%	44.64%
Final	84	6	31	90	93.33%	48.28%	63.64%

类别表：

Category	#Case	#Apps	#Held	Prec(%)	Recall(%)	F1(%)
AI Chat	19	10	5	100.0	64.3	78.3

Bluetooth	3	2	0	100.0	66.7	80.0
Cloud Drive	18	11	5	88.9	61.5	72.7
Code Hosting	17	8	4	100.0	61.5	76.2
Collaboration	18	9	4	100.0	35.7	52.6
Content	9	4	0	100.0	25.0	40.0
Content Transform	7	2	0	100.0	71.4	83.3
E2E	24	1	0	100.0	50.0	66.7
Email	19	8	5	71.4	35.7	47.6
File Structure	12	4	0	100.0	45.5	62.5
IM	15	7	4	88.9	72.7	80.0
Meeting	15	6	3	66.7	33.3	44.4
Screen	8	2	0	100.0	37.5	54.5
Steganography	3	2	0	100.0	0.0	0.0
Technical Forum	14	10	4	100.0	40.0	57.1
Transfer	7	4	0	100.0	66.7	80.0
Virtual Machine	3	3	0	100.0	0.0	0.0
Overall	211	88	33	93.3	48.3	63.6

评估解读：

- Deterministic precision 为 100%，说明日志确定性链路在当前数据中非常保守，没有产生 FP。
- Triage recall 为 100%，说明需要送 VLM 的候选没有漏掉正例。
- Final precision 为 93.3%，说明 VLM 复核显著降低了 FP。
- Final recall 仅 48.3%，说明 VLM 完成态判断和后处理策略过严，是下一步主要优化对象。

1.18 关键实现风险

风险	现状	建议
VLM 召回不足	邮件、会议、协作工具场景中，大量正例被判为未完成外发。	对 FN 按类别抽样复盘，分别调整邮件发送成功、AI 内容暴露、会议共享完成态 prompt。
类别推断依赖 case 名	表格生成器当前根据样本目录名推断 category/app/held。	增加 dataset manifest，显式记录类别、应用、是否 hold-out、正负标签。
Neo4j 未完全落地	当前图结构主要存在于 JSON bundle 和内存对象中。	增加 graph writer，把 File、Event、Window、FrontendApp、ContentArtifact 和证据边写入 Neo4j。
目录迁移风险	顶层目录仍有历史编号，直接移动会影响 sys.path 和运行脚本。	先建立统一包入口和兼容 import，再迁移目录。
多轮追踪未闭环	已有多跳 lineage，但 follow-up 自动补跑尚未形成状态机。	实现 closed、terminal、needs_followup，限制 round 和窗口数量。

1.19 后续计划

后续工作按优先级建议如下：

1. 分析全量 VLM 的 90 个 FN，输出按类别、sink、VLM reason 的错误分布。
2. 针对 Email、Meeting、Collaboration 三类低召回场景重写完成态 prompt 和后处理规则。
3. 增加 dataset manifest，固定 benchmark 表格口径。
4. 将 EventCorrelator 的 analysis_windows 接入 FrameAnalyzer 的真实窗口抽帧，而不是仅在 benchmark 中模拟。
5. 实现 Neo4j graph writer，落地 Process -> Event -> File/Window/FrontendApp 和 artifact 边。
6. 实现 artifact follow-up 多轮补跑，覆盖 file->content、content->file、file->file、content->content 四类转换。

7. 在目录迁移前建立统一 Python package 入口，减少顶层目录名对运行脚本的耦合。

1.20 结论

h 分支已经完成从单点规则修复到系统性架构补强的过渡。日志侧具备确定性检测和高召回 VLM gate；视觉侧具备 live VLM 复核和结果后处理；事件关联侧新增 EventCorrelator，补齐前台应用识别、敏感窗口、多跳 lineage 和 correlation bundle；评估侧具备全量 NAS benchmark 和论文表格输出。

当前系统的主要短板已经明确：不是候选发现不足，而是 VLM 最终判定召回不足。后续优化应围绕完成态 prompt、类别化后处理、数据集 manifest 和图数据库写回展开。该分支为这些工作提供了可运行、可评估、可追踪的工程基础。